

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин

Лабораторная работа №3 по курсу
«Операционные системы и системное программирование»
на тему
«Взаимодействие и синхронизация процессов»

Выполнил:

ст.гр. 450501
Лебедь М.В.

Проверил:

ст. преп. каф. ЭВМ
Поденок Л.П.

МИНСК 2026

1 УСЛОВИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Управление дочерними процессами и упорядочение вывода в `stdout` от них, используя сигналы `SIGUSR1` и `SIGUSR2`.

Действия родительского процесса

По нажатию клавиши «+» родительский процесс (P) порождает дочерний процесс (C_k) и сообщает об этом.

По нажатию клавиши «-» P удаляет последний порожденный C_k, сообщает об этом и о количестве оставшихся.

При вводе символа «\» выводится перечень родительских и дочерних процессов.

При вводе символа «к» P удаляет все C_k и сообщает об этом.

По нажатию клавиши «q» P удаляет все C_k, сообщает об этом и завершается.

Действия дочернего процесса

Дочерний процесс во внешнем цикле заводит будильник (`nanosleep(2)`) и входит в вечный цикл, в котором заполняет структуру, содержащую пару переменных типа `int`, значениями {0, 0} и {1, 1} в режиме чередования. Поскольку заполнение не атомарно, в момент срабатывания будильника в структуре может оказаться любая возможная комбинация из 0 и 1.

При получении сигнала от будильника проверяет содержимое структуры, собирает статистику и повторяет тело внешнего цикла.

Через заданное количество повторений внешнего цикла (например, через 101) дочерний процесс выводит свои PPID, PID и 4 числа — количество разных пар, зарегистрированных в момент получения сигнала от будильника.

Вывод осуществляется в одну строку и должен быть отформатирован.

Следует подобрать интервал времени ожидания и количество повторений внешнего цикла, чтобы статистика была значимой (101, как вариант).

Сообщения выводятся в `stdout`.

Сообщения процессов должны содержать идентифицирующие их данные, чтобы можно было фильтровать вывод утилитой `grep`.

2 ОПИСАНИЕ АЛГОРИТМОВ И РЕШЕНИЙ

2.1 Алгоритм работы родительского процесса

Для управления жизненным циклом дочерних процессов и синхронизации их работы реализован родительский процесс. Алгоритм его работы:

- 1) Инициализация обработчика сигнала SIGUSR1 для организации очереди вывода дочерних процессов.
- 2) Получение пути к исполняемому файлу дочернего процесса из переменной окружения CHILD_PATH или использование значения по умолчанию.
- 3) Инициализация структуры хранения PID дочерних процессов (стек).
- 4) Переход в цикл обработки пользовательского ввода.
- 5) Ожидание команды пользователя (+, -, l, k, q) с последующей очисткой входного буфера.
- 6) При вводе «+» создаётся дочерний процесс с помощью fork и execve, его PID сохраняется в стеке.
- 7) При вводе «-» завершается последний созданный дочерний процесс и обновляется стек.
- 8) При вводе «l» выводится информация о родительском и дочерних процессах.
- 9) При вводе «k» выполняется завершение всех дочерних процессов.
- 10) При получении SIGUSR1 от дочерних процессов организуется их очередь и поочерёдно разрешается вывод через SIGUSR2.
- 11) При вводе «q» завершаются все дочерние процессы и осуществляется выход из цикла.
- 12) После завершения работы освобождаются ресурсы и программа корректно завершается.

2.2 Алгоритм работы дочернего процесса

Дочерний процесс предназначен для моделирования неатомарных операций и сбора статистики состояний при асинхронных событиях. Алгоритм его работы:

- 1) Инициализация обработчиков сигналов SIGALRM и SIGUSR2.
- 2) Инициализация структуры для хранения фиксируемых состояний.
- 3) Переход в цикл с заданным числом повторений (REPEATS).
- 4) На каждой итерации создаётся вспомогательный процесс, который по истечении времени отправляет сигнал SIGALRM.
- 5) В основном процессе выполняется цикл неатомарной записи значений {0,0} и {1,1} в структуру.
- 6) При получении SIGALRM фиксируется текущее состояние структуры.
- 7) Зафиксированное значение сохраняется в массив уникальных комбинаций.
- 8) После завершения всех итераций блокируется сигнал SIGUSR2 и отправляется SIGUSR1 родителю о готовности к выводу.
- 9) Ожидается разрешение на вывод с использованием sigsuspend.
- 10) После получения SIGUSR2 выводятся PPID, PID и статистика состояний.
- 11) По завершении вывода отправляется сигнал родителю и процесс завершается.

3 ОПИСАНИЕ ФУНКЦИОНАЛЬНОЙ СТРУКТУРЫ ПРОЕКТА

3.1 Состав и назначение модулей

1) `parent.c` – реализует логику родительского процесса, который управляет созданием, удалением и координацией работы дочерних процессов.

2) `functions.h` – заголовочный модуль, содержащий определение структур, констант и прототип функции.

3) `child.c` – содержит реализацию поведения дочернего процесса: генерацию случайной статистики, взаимодействие с родителем через сигналы, а также механизм таймера.

3.2 Используемые структуры данных

Для хранения идентификаторов процессов применяется стек PID, реализованный на основе односвязного списка, что обеспечивает управление порядком создания и удаления процессов (LIFO). Операции добавления и удаления элементов выполняются динамически с использованием выделения и освобождения памяти.

Дополнительно в обработчике сигналов используется очередь PID, реализованная на основе кольцевого массива фиксированного размера. Данная структура применяется для последовательной обработки запросов на вывод от дочерних процессов и обеспечивает корректную синхронизацию их выполнения.

4 ОПИСАНИЕ ПОРЯДКА СБОРКИ И ИСПОЛЬЗОВАНИЯ

4.1 Сборка проекта

Сборка проекта осуществляется при помощи make.

```
[Mixeld@archlinux LAB_3]$ make
mkdir -p build/debug
gcc -W -Wall -Wextra -std=c11 -pedantic -Wno-unused-parameter
-Wno-unused-variable -c src/parent.c -o build/debug/parent.o
gcc -W -Wall -Wextra -std=c11 -pedantic -Wno-unused-parameter
-Wno-unused-variable -c src/child.c -o build/debug/child.o
gcc build/debug/parent.o build/debug/child.o -o build/debug/lab3
[Mixeld@archlinux LAB_3]$
```

4.2 Использование программы

```
[Mixeld@archlinux LAB_3]$ make run
./build/debug/lab3
[p] START!
'+' - create
'-' - delete last
'l' - list
'k' - kill all
'q' - quit
[p] Created child with PID = 107733, Total children: 1
[C 107733] FORKED SUCCESSFULLY! About to enter
child_main_loop...
[C 107733] Cycle 1/50 completed. Waiting for parent
permission...
[P] Child 107733 is ready to report. Authorizing print...
[C] PPID=107727, PID=107733, stats: {0,0}=492, {0,1}=499, {1,0}
=511, {1,1}=498
[p] Created child with PID = 107734, Total children: 2
[C 107734] FORKED SUCCESSFULLY! About to enter
child_main_loop...
[C 107734] Cycle 1/50 completed. Waiting for parent
permission...
[P] Child 107734 is ready to report. Authorizing print...
[C] PPID=107727, PID=107734, stats: {0,0}=522, {0,1}=512, {1,0}
=452, {1,1}=514
[C 107733] Cycle 2/50 completed. Waiting for parent
permission...
[P] Child 107733 is ready to report. Authorizing print...
[C] PPID=107727, PID=107733, stats: {0,0}=474, {0,1}=518, {1,0}
=496, {1,1}=512
```

```

[C 107734] Cycle 2/50 completed. Waiting for parent
permission...
[P] Child 107734 is ready to report. Authorizing print...
[C] PPID=107727, PID=107734, stats: {0,0}=500, {0,1}=515, {1,0}
=497, {1,1}=488
[C 107733] Cycle 3/50 completed. Waiting for parent
permission...
[P] Child 107733 is ready to report. Authorizing print...
[C] PPID=107727, PID=107733, stats: {0,0}=491, {0,1}=527, {1,0}
=503, {1,1}=479
[C 107734] Cycle 3/50 completed. Waiting for parent
permission...
[P] Child 107734 is ready to report. Authorizing print...
[C] PPID=107727, PID=107734, stats: {0,0}=513, {0,1}=483, {1,0}
=514, {1,1}=490
[p] Parent PID: 107727
[p] Children PIDs (2 total): 107733 107734
[C 107733] Cycle 4/50 completed. Waiting for parent
permission...
[P] Child 107733 is ready to report. Authorizing print...
[C] PPID=107727, PID=107733, stats: {0,0}=500, {0,1}=506, {1,0}
=468, {1,1}=526
[C 107734] Cycle 4/50 completed. Waiting for parent
permission...
[P] Child 107734 is ready to report. Authorizing print...
[C] PPID=107727, PID=107734, stats: {0,0}=543, {0,1}=487, {1,0}
=487, {1,1}=483
[C 107733] Cycle 5/50 completed. Waiting for parent
permission...
[P] Child 107733 is ready to report. Authorizing print...
[C] PPID=107727, PID=107733, stats: {0,0}=471, {0,1}=517, {1,0}
=488, {1,1}=524
[C 107734] Cycle 5/50 completed. Waiting for parent
permission...
[P] Child 107734 is ready to report. Authorizing print...
[C] PPID=107727, PID=107734, stats: {0,0}=528, {0,1}=500, {1,0}
=456, {1,1}=516
[C 107733] Cycle 6/50 completed. Waiting for parent
permission...
[P] Child 107733 is ready to report. Authorizing print...
[C] PPID=107727, PID=107733, stats: {0,0}=575, {0,1}=470, {1,0}
=501, {1,1}=454
[C 107734] Cycle 6/50 completed. Waiting for parent
permission...
[P] Child 107734 is ready to report. Authorizing print...
[C] PPID=107727, PID=107734, stats: {0,0}=496, {0,1}=545, {1,0}
=453, {1,1}=506
[C 107733] Cycle 7/50 completed. Waiting for parent
permission...

```



```
[P] Child 107733 is ready to report. Authorizing print...
[C] PPID=107727, PID=107733, stats: {0,0}=480, {0,1}=511, {1,0}
=502, {1,1}=507
[C 107734] Cycle 7/50 completed. Waiting for parent
permission...
[P] Child 107734 is ready to report. Authorizing print...
[C] PPID=107727, PID=107734, stats: {0,0}=503, {0,1}=476, {1,0}
=523, {1,1}=498
[p] kill all 2 children
[p] Deleting child with PID = 107734
[C 107734] Terminated by signal.
ТЕРМИНАЛ
[p] Deleting child with PID = 107733
[C 107733] Terminated by signal.
ТЕРМИНАЛ
[p] ALL CHILDREN KILLED
q
[p] Quitting...
NO MORE CHILDREN
ТЕРМИНАЛ
```

5 ОПИСАНИЕ МЕТОДОВ ТЕСТИРОВАНИЯ И РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ ПРОГРАММЫ

5.1 Методы тестирования программы

Программа тестировалась вручную, а также через утилиту valgrind.

5.2 Результаты тестирования

Результаты ручного тестирования

```
[bazelin@archlinux lab03]$ export CHILD_PATH=./build/debug/child
[bazelin@archlinux lab03]$ ./build/debug/parent
[+] add
[-] kill last
[k] kill all
[l] list
[q] quit
+
PID: 167044 started
PPID 167038 | PID: 167044 | 00:8 01:19 10:34 11:39
PID: 167302 started
+
PID: 167319 started
PPID 167038 | PID: 167302 | 00:9 01:20 10:22 11:49
PPID 167038 | PID: 167319 | 00:12 01:23 10:28 11:37
l
parent PPID: 17234
child_00 PID: 167302
child_01 PID: 167319
k
kill all child is successfull
q
list of child is empty
```

Результат тестирования через valgrind

```
valgrind --leak-check=full --show-leak-kinds=all -s
./build/debug/parent
==167038==
==167038== HEAP SUMMARY:
==167038==      in use at exit: 0 bytes in 0 blocks
==167038==    total heap usage: 3 allocs, 3 frees, 2,064 bytes
allocated
==167038==
==167038== All heap blocks were freed -- no leaks are possible
```

```
==167038==  
==167038== ERROR SUMMARY: 0 errors from 0 contexts (suppressed:  
0 from 0)
```